I'll keep preprocessing (cells/initial portals/etc.) minimal and focus on the moving parts.

0) The picture: global search + local refinement "swimlanes"

Think of 3 concurrent engines:

```
sql

(1) Global Overlay Search (Forward)      (2) Global Overlay Search (Backward)
      OpenF (PQ)                               OpenB (PQ)
      BinsF[v]                                 BinsB[v]
      expands labels ℓF                         expands labels ℓB
                \                               /
                 \    (meet at same portal v)  /
                  --->  Join(ℓF, ℓB)  ---> Candidate full path

(3) Local Cell Engine(s) (per cell, per source portal, per direction)
      LocalPQ[cell][portal][dir]
      k-hop limited incremental expansion
      discovers portal→portal connections
      inserts/updates overlay edges
      immediately spawns new global labels using those new edges
```

Key point: Bidirectional search is **on the overlay**, not "inside each cell". Th Copier le code work is the **Local Cell Engine**, which is incremental and budgeted.

This architecture is the "No-boundary → Post-boundary repair" spirit: start with incomplete intra-cell knowledge, refine only where needed 2024 A Universal Scheme for Dyn... , plus anytime reuse ideas (pruned labels reused across rounds) 2024 Apex Efficient Anytime App... , and bidirectional match/join mechanics 2025 Resource Constrained Pathf... .

1) The minimum set of notations you must track in the while-loop

1.1 Label (global)

A **label** ℓ is a partial path in the overlay:

- `ℓ.v` : current overlay node (portal)
- `ℓ.g[1..4]` : accumulated cost vector from start to ℓ.v (forward) or from ℓ.v to goal (backward)
- `ℓ.parent` pointer, `ℓ.parentEdge` : for reconstruction
- `ℓ.lbScore` : admissible lower bound on final Chebyshev score

Forward vs Backward semantics

- **Forward label ℓF** represents a real path $s \to \ellF.v$
- **Backward label ℓB** represents a real path $\ellB.v \to t$
  - Implementation: backward expands on the **reversed overlay graph**, but the stored `g` is still "cost from current node to target" (so sums are consistent when joining).

1.2 Overlay edges

Overlay edge `e: u -> v` stores:

- `e.cost[1..4]` : vector cost of the represented base-graph subpath
- `e.pathPtr` : optional pointer for decompression (sequence of base vertices)
- `e.type ∈ {INTER, SHORTCUT}`

Cost update rule when expanding:

$$g(\text{child}) = g(\text{parent}) + c(e)$$

(component-wise sum)

1.3 Local cell engine state

For each **cell i** and **source portal p**, in each direction `dir` :

- `LocalPQ[i][p][dir]` holds local states `σ`
- local state `σ` includes:
  - `σ.x` : a vertex inside the cell
  - `σ.gLocal[1..4]` : cost from portal p to σ.x (in `dir`)
  - `σ.hops` : number of intra-cell edges used so far
  - `σ.parent` : local parent for reconstructing p→...→x

This engine is **persistent** across calls: you don't restart; you "continue expanding" as k grows.

2) The core while-loop (global overlay search) — fully explained

I'll write it as a "flow diagram + exact meaning".

2.1 The loop skeleton

python

```
while OpenF or OpenB not empty:

  dir = chooseDirection(OpenF, OpenB)      # e.g., smallest min lbScore

  ℓ = popMin(Open[dir])                    # label with smallest key

  # --- (A) Feasibility / pruning using box-bounds ---
  compute LBrem/UBrem from hops-to-target (cheap)
  if BoxInfeasible(ℓ): continue
  if ℓ.lbScore ≥ UBscoreBest: continue

  # --- (B) Insert label into per-node bins (approx compression) ---
  if not KeepInBins(ℓ, Bins[dir][ℓ.v]):
        storeIntoPRUNED(ℓ) optionally
        continue

  # --- (C) Bidirectional meeting at same portal ---
  for ℓ2 in CandidateJoins(ℓ, Bins[opp][ℓ.v]):
        π = Join(ℓ, ℓ2)
        if feasible(π) and score(π) improves: update SOL, UBscoreBest

  # --- (D) Ensure enough intra-cell shortcut edges exist BEFORE expansion ---
  if NeedShortcuts(ℓ.v, dir):
        LocalRefineFromPortal(ℓ, dir)      # k-hop within cell, inserts edges
        # IMPORTANT: new edges may create new children immediately

  # --- (E) Expand ℓ over ALL CURRENT overlay edges in direction dir ---
  for each overlay edge e usable in dir from ℓ.v:
        ℓchild = Extend(ℓ, e)
        push(Open[dir], ℓchild)
```

Now I'll explain each block with the "why" and the exact mechanics.

2.2 (A) Feasibility pruning is interval-based (critical for double-sided bounds)

You compute remaining bounds (cheap):

- `hops = cellHopLowerBound(cell(ℓ.v), targetCell)`
- `LBrem[i] = hops * minEdge[i]` (safe)
- `UBrem[i] = hops * maxEdge[i]` (safe but loose; you can even omit UBrem and use `U[i]` directly)

Then for each dimension i:

- `low_i = max(L[i], ℓ.g[i] + LBrem[i])`
- `high_i = min(U[i], ℓ.g[i] + UBrem[i])` (or simply `U[i]`)

If any `low_i > high_i`, then **no completion can satisfy the box** → prune.

This pruning is one of the few dominance-like things that stays safe when you have lower limits (classic dominance can fail under lower bounds) 2024 A two-stage method for dou...

2.3 (B) "KeepInBins" = your bounded-memory approximation mechanism

At portal `v = ℓ.v`, you keep only a small number of representatives:

- Compute `BinKey(ℓ.g, δ)`
- If the bin is empty: keep `ℓ`
- If bin is occupied: keep the "better" one (by lbScore or feasibility-first tie-break)

Important: Binning is where you can lose completeness *in a single round*. That's why:

- δ decreases over rounds
- pruned labels can be reused (A-A*pex idea) 2024 Apex Efficient Anytime App...

2.4 (C) Bidirectional meeting: ℓ + ℓ_2 at same portal

Let `v = ℓ.v`.

If ℓ is forward and ℓ_2 is backward (or vice versa), and both end at the same `v`, then:

- total cost vector:

$$g_{\text{tot}} = g(\ell) + g(\ell_2)$$

- feasibility:

$$L \le g_{\text{tot}} \le U$$

- score:

$$Score(g_{\text{tot}}) = \max_i w_i |g_{\text{tot},i} - C_i|$$

If feasible and improves best solution, update:

- `SOL` archive
- `UBscoreBest` (global best known score)

This is the same fundamental "join forward/backward partials at meeting state" idea used by constrained bidirectional A* frameworks 2025 Resource Constrained Pathf... .

Where do meeting points occur?

Only at overlay nodes (portals).
Not inside cells (unless you temporarily treat interior vertices as portals, which is a refinement option).

OK

3) The local k-hop within a cell: how shortcuts are created and inserted

This is the part you explicitly want checked and explained.

3.1 When does local refinement run?

When you pop a global label ℓ at a portal `p=ℓ.v` and you decide:

- "I don't have enough intra-cell options out of p"
  OR
- "I keep failing to connect p to needed portals"
  OR
- "this portal is hot (expanded many times) but overlay is missing edges"

Then:

```scss
LocalRefineFromPortal(ℓ, dir)
```

Copier le code

Crucial improvement (recommended)

Pass the **residual upper budget** of this particular label:

$$U^{res}[i] = U[i] - \ell.g[i]$$

Because all costs are nonnegative, any local path segment with

`gLocal[i] > Ures[i]` **cannot** be part of any feasible completion of ℓ.

So you can prune local exploration very strongly, without breaking correctness.

This is one of the biggest practical wins.

3.2 Local engine: incremental k-hop multiobjective expansion

LocalRefineFromPortal(ℓ, dir) detailed behavior

Inputs:

- `cell i = cell(ℓ.v)`
- `source portal p = ℓ.v`
- `dir` (F or B)
- `k` (max hop depth allowed this round)
- `stepBudget` (#pop operations allowed this call)
- `Ures = U - ℓ.g` (component-wise)

Persistent state:

- `LocalPQ[i][p][dir]` already has some frontier from previous calls

Algorithm:

```bash
LocalRefineFromPortal(ℓ, dir):

  i = cell(ℓ.v)
  p = ℓ.v
  Ures = U - ℓ.g

  repeat stepBudget times:
      if LocalPQ[i][p][dir] empty: break

      σ = LocalPQ[i][p][dir].popMin()
```

```
if σ.hops > k: continue

# (1) upper-budget pruning (safe!)
if exists dim j: σ.gLocal[j] > Ures[j]: continue

# (2) local binning / compression (optional but recommended)
if LocalBinningReject(i,p,σ): continue

for each intra-cell edge (σ.x -> y) that stays in cell i in direction dir:
      σ2.x = y
      σ2.gLocal = σ.gLocal + c(σ.x,y)
      σ2.hops = σ.hops + 1
      σ2.parent = σ
      push(LocalPQ, σ2)

      if y is an ACTIVE portal q and q != p:
           RegisterShortcutCandidate(p,q,σ2, dir)
```

Copier le code

Cost updates inside the cell

Exactly the same additive update rule:

$$gLocal(\sigma_2) = gLocal(\sigma) + c(\sigma.x, y)$$

So the shortcut edge (p→q) gets:

- `cost = σ2.gLocal`
- `pathPtr = reconstruct local parents p→…→q`

3.3 RegisterShortcutCandidate: how you insert edges without losing necessary tradeoffs

When you discover a candidate portal→portal connection p→q:

You should **NOT keep only one** edge per (p,q).
Because with box constraints + Chebyshev objective, different tradeoffs can matter.

So implement:

- for each pair (p,q), maintain a small set `Shortcuts[p,q]`

- prune with **local binning** (same δ scheme) OR keep up to `Kedge` nondominated-ish representatives
- store each as a separate overlay edge

This is consistent with overlay ideas that keep only a reduced set of "relevant" boundary connections 2016 COLA Effective Indexing fo... .

4) The most important subtlety: what happens when a new shortcut is inserted?

If you insert a new overlay edge `p -> q`, you must ensure it is **actually used** by the global search.

There are two safe ways:

Option A (simple, safe, fast): "spawn children immediately from the current ℓ"

When LocalRefine is called *while expanding* ℓ *at* p:

- every time you insert a new shortcut p→q
- you ~~immediately create~~:

$$\ell' = Extend(\ell, p \to q)$$

- and push ℓ' into Open (same direction)

This ensures you do not need decrease-key or re-expansion logic.

✅ This is the easiest way to get correctness.

Option B (more complete): "spawn from all representative labels at p"

Sometimes the new shortcut is discovered while expanding one label ℓ, but it could help another label at p too.

Because you keep only a small number of labels per portal (Bins), you can do:

- For each `ℓrep in Bins[dir][p]`:
  - push `Extend(ℓrep, p→q)`

This costs O(#bins at p) per new edge — usually small.

✅ Recommended for better anytime performance.

5) How backward local refinement works (don't ignore this)

If the overlay is directed, backward expansion needs **incoming** edges.

So for backward direction:

- global backward expands along reversed overlay edges
- local refinement must also be able to "grow shortcuts in the direction needed"

Two choices:

Choice 1 (cleanest): keep two local engines

- `LocalPQ[i][p][F]` expands forward intra-edges
- `LocalPQ[i][p][B]` expands backward intra-edges (i.e., along reversed intra-edges)

Choice 2 (if graph is undirected/symmetric): reuse forward shortcuts

If base edges are undirected or you store both directions when you discover p↔q, then you can reuse.

6) What about "Post-boundary repair" and intra-cell shortcuts being wrong?

This is not a bug; it's expected.

A shortest (or best tradeoff) path between two boundary vertices inside a cell may actually **leave the cell and re-enter**. That is exactly what the PSP "No-boundary vs Post-boundary" discussion is about 2024 A Universal Scheme for Dyn...

In your algorithm, this matters like this:

- local k-hop only explores **within the cell**
- so it may miss a better p→q option that goes outside

But the global overlay search can still find it as a multi-cell route via other portals, *even without repair* — as long as the portal set is rich enough.

Repair becomes necessary if:

- your portal set is very sparse, and
- you rely heavily on direct "same-cell shortcut edges"

So, if you keep portals sparse (your Option 2), I recommend you do **lightweight PostBoundaryRepair only for portal pairs that appear in best solutions**.

7) Checklist: if you implement these rules, the reasoning is consistent

Soundness (no fake solutions)

- Every overlay edge corresponds to a real base path (inter-edge or local-discovered path).
- Every label corresponds to a real concatenation of base paths.
- Every joined solution corresponds to a real path s→…→v→…→t (reconstructable).

Feasibility correctness

- You only accept solutions after checking $L \le g \le U$.
- You prune partial labels only with *safe interval infeasibility* tests (good under lower limits) 2024 A two-stage method for dou… .

Convergence to feasible solutions (in the limit)

You will converge to a feasible solution if one exists, provided that:

- portals are eventually expanded enough to include required boundary crossings
- k grows enough that needed intra-cell connectivity is discovered (or global overlay finds multi-cell alternatives)
- δ shrinks enough (or you allow enough bins per portal) so you don't permanently discard the needed partial label

Anytime reuse of pruned labels helps prevent losing progress across rounds

2024 Apex Efficient Anytime App…

8) Two refinements I strongly recommend (directly about your "while loop + k-hop")

Refinement 1: residual-budget local pruning

When local refinement is triggered by label ℓ at portal p, always pass:

$$U^{res} = U - \ell.g$$

and prune any local state whose `gLocal > Ures` component-wise.

This makes local refinement *targeted* and avoids exploding local search.

Refinement 2: "on-edge-insert spawn" so you never miss newly learned shortcuts

Whenever you insert a new shortcut `p->q`, immediately generate new global children from:

- the current ℓ being expanded (minimum)
- optionally all Bins[dir][p] representatives (better anytime)

Without this, dynamic overlay updates can be silently ignored by the global search.

If you want, I can now produce the same content as a **single clean flowchart** (Mermaid code), *focused only on the while-loop and LocalRefine blocks*, so you can paste it into a document and "see" the algorithm structure instantly.

Sources

OK for the Mermaid code

Réflexion durant 8s >

Below is a "notation + function semantics + main loop mechanics" explanation that's meant to be **exhaustive where it matters** (labels ℓ/$\ell 2$, k-hop intra-cell growth, portal

shortcuts, bidirectional matching), while keeping initialization/preprocessing lightweight as you requested.

I'll reference the ideas you anchored on:

- **Overlay on boundary/portals + keep only non-dominated connections** (COLA overlay concept). 2016 COLA Effective Indexing fo...
- **Double-sided constraints break classic dominance intuition** (so feasibility tests and bounds dominate pruning). 2024 A two-stage method for dou...
- **Bidirectional constrained search + joining forward/backward partials at a meet node** (matching). 2025 Resource Constrained Pathf...
- **Min–Max RCSP incremental/bidirectional ideas and the importance of avoiding expensive cycle checks** (context for feasibility-first + incremental exploration). 2025 Answering_Min-Max_Resource...
- MOS background: approximate/anytime sets, approximate dominance, etc. 2025 Multi-Objective Search Alg...

0) What graph are we actually searching?

You have two graphs at runtime:

(A) Base graph G = (V,E)

Original road network with 4 additive attributes per edge:

$$c(e) = (c_1, c_2, c_3, c_4) \quad \text{all minimized}$$

Path accumulation is additive: $g(\pi) = \sum c(e)$.

(B) Overlay graph Go = (Vo,Eo) on portals

- **Vo** = portals (a subset of boundary vertices).
- **Eo** contains:
  1. **Inter-cell edges**: original edges that cross cell boundaries (cheap & exact).
  2. **Intra-cell shortcut edges**: learned gradually by the *k-hop local engine* inside each cell.

This is the same "overlay edges represent paths in the original graph" idea as COLA's overlay definition (edges stand for underlying paths; multiple edges between same endpoints can exist). 2016 COLA Effective Indexing fo...

1) Constraints & objective you optimize

Box constraints (double-sided, per dimension)

$$L \le g(\pi) \le U$$

This is the "min–max / doubly constrained" setting where **lower bounds matter**, and standard dominance pruning becomes unreliable (a dominated partial path may be the only one that can still reach the minimum). 2024 A two-stage method for dou...

2025 Answering_Min-Max_Resource...

Score to minimize (your "Chebyshev-to-center")

Let $C = (L + U)/2$. With weights w_i:

$$Score(\pi) = \max_i w_i \cdot |g_i(\pi) - C_i|$$

We maintain a **best-so-far upper bound** UB^* = best Score found among feasible complete solutions so far (anytime behavior).

2) Labels: what is ℓ and what is ℓ2?

A **label** is a *search state representing a partial path* on the overlay (and, implicitly, on the base graph).

Forward label ℓ (from s)

pgsql

```
ℓ = {
  v:     current overlay node (portal or s/t if injected),
  g[4]:  accumulated 4D cost from s to v along chosen overlay edges,
  prev:  pointer to previous label (for path reconstruction),
  prev_e: which overlay edge was taken,
  meta:  {round r, etc.}
  LB:    admissible lower bound on best achievable final Score from this label
}
```

Copier le code

Backward labels are symmetric: they accumulate from t backward.

Extension operation: ℓ2 = Extend(ℓ, e)

If overlay edge `e = (v -> u)` has vector cost `c_e[4]`, then:

$$g(\ell_2) = g(\ell) + c_e$$

and `ℓ2.v = u`, with parent pointers updated.

That's the only "cost update on expansion": **vector addition**.

3) The two essential bound types used in pruning

Because double-sided constraints weaken dominance pruning, you should rely on two tests first:

(A) Feasibility reachability bounds (box feasibility)

For each dimension i, assume you have a remaining-cost interval from node v to target:

$$\Delta_i \in [LB_i(v), UB_i(v)]$$

Then any completion must satisfy:

$$g_i(\ell) + LB_i(v) \le g_i(\pi) \le g_i(\ell) + UB_i(v)$$

Infeasible w.r.t upper bound if:

$$g_i(\ell) + LB_i(v) > U_i$$

Infeasible w.r.t lower bound if:

$$g_i(\ell) + UB_i(v) < L_i$$

This idea—"use lower limits for pruning / reductions, but dominance no longer works cleanly"—is exactly the warning emphasized in doubly constrained work.

2024 A two-stage method for dou...

(B) Score lower bound LBscore(ℓ)

Using the same intervals, compute the **best possible** Chebyshev deviation you could still achieve:

- Let $X_i = [g_i(\ell) + LB_i(v),\ g_i(\ell) + UB_i(v)]$.
- If $C_i \in X_i$, minimum deviation is 0; else distance to nearest endpoint.
- Then $LBscore(\ell) = \max_i w_i \cdot d_i$.

Prune if:

$$LBscore(\ell) \ge UB^*$$

This is your "anytime convergence driver": once you have *any* feasible solution, UB^* shrinks and pruning becomes very strong.

4) "Sparse portals early, exact only in the limit" (Option 2) — what it means precisely

You start with:

- **Few portals** per cell.
- **Few intra-cell shortcut edges** per portal pair.
- **Coarse label compression** (binning / ε-like approximation).

Then as rounds progress you relax approximations:

- Add more portals (activate more boundary vertices).
- Expand local k-hop searches further, producing better shortcuts.
- Refine the label compression resolution (smaller bins / smaller ε).
- Eventually, in the limit (if you let budgets go to infinity and stop compressing), you converge to the exact overlay of all boundary vertices + full intra-cell connectivity.

This is the same "approximate early, refine later" philosophy used in approximate/anytime MOS and overlay-based approaches. 2025 Multi-Objective Search Alg...

2016 COLA Effective Indexing fo...

5) Core data structures in the main loop

Global search (overlay)

For each direction d ∈ {F,B}:

- `Open_d` : priority queue ordered by `LBscore` (or lexicographic surrogate).
- `Bins_d[v]` : compressed set of "kept" labels at overlay node v.
  - Key idea: keep only a small number per node (per bin / per region).
- `PrunedPool_d` : labels you *did not keep* only because of compression, not because they were infeasible; reused next round (anytime reuse).

Local engines (within each cell)

For each cell `cell_id` and each portal `p` in that cell:

- `LocalOpen[cell_id,p]` : a queue for local exploration states (local labels).
- `LocalSeen[cell_id,p]` : local binning / pruning structure.
- `ShortcutStore[cell_id]` : best-known shortcuts among portals in that cell:

- `ShortcutStore[cell][p][q] = {costVector, underlyingPathPointer, qualityStamp}`

6) The key: what happens in the main while-loop?

I'll describe *exactly* what the loop is doing at the level of ℓ/ℓ2 and k-hop.

Step 0 — pick direction

At each iteration, pick the direction (forward/backward) whose queue has the smallest current key:

- e.g., compare `min(OpenF).LBscore` vs `min(OpenB).LBscore`.

This mirrors bidirectional A*-style "expand the most promising frontier" behavior.

2025 Resource Constrained Pathf...

Step 1 — pop a label ℓ

makefile

```
ℓ = Open_d.pop_min()
v = ℓ.v
```

Copier le code

Step 2 — feasibility pruning (box reachability)

Before any expansion:

1. Compute remaining bounds `[LB_i(v), UB_i(v)]`.
2. If ℓ cannot possibly be completed into [L,U], discard.

This is your first mandatory pruning layer in double-sided constraints.

2024 A two-stage method for dou...

Step 3 — upper-bound pruning (anytime)

If `LBscore(ℓ) ≥ UB*`, discard.

Step 4 — compression / binning at node v

You maintain only a few labels per node in `Bins_d[v]`.

KeepInBins(ℓ, v):

- Compute normalized coordinates of $g(\ell)$ in the [L,U] box (clipped).
- Compute bin index (round-dependent resolution).
- If the bin is empty → keep ℓ.
- If occupied → keep the label with smaller LBscore (or better "potential"), push loser to `PrunedPool_d`.

This is your practical mechanism to prevent label explosion (approximate MOS style).

2025 Multi-Objective Search Alg...

Important subtlety for double-sided constraints:
You **must not** delete labels purely by classical dominance (g smaller) because that can remove the only label capable of reaching the lower bound. Binning avoids that trap by keeping *representatives across the box.* 2024 A two-stage method for dou...

Step 5 — bidirectional meet / match (how merging works)

This is the "meeting point" logic you asked about.

When ℓ is **accepted** into `Bins_d[v]`, you attempt joins with labels stored at the same node in the opposite direction:

```
for each ℓ_opp in Bins_opposite[v]:
    g_total = g(ℓ) + g(ℓ_opp)    # vector addition
    if L <= g_total <= U:
        Score = ChebyshevToCenter(g_total)
        update UB* and archive if Score improves / adds diversity
```

Copier le code

This is exactly the constrained bidirectional matching idea: complete paths are formed by joining forward/backward partials at the same state/node. 2025 Resource Constrained Pathf...

Do we do bidirectional search "within each cell"?

No: the *bidirectional* part is global on the overlay.

Inside a cell, your k-hop local engines can be unidirectional per portal seed (simpler), and you still get global bidirectional benefits by joining at overlay nodes.

Step 6 — expand ℓ on existing overlay edges (creates $\ell 2$)

Now you generate successors using the overlay adjacency list.

For each overlay edge $e = (v \to u)$ with cost vector c_e:

```csharp
ℓ2 = ℓ
ℓ2.v = u
ℓ2.g = ℓ.g + c_e
ℓ2.prev = ℓ
ℓ2.prev_e = e
compute LBscore(ℓ2)
push ℓ2 into Open_d
```

Copier le code

That's the full $\ell \to \ell 2$ creation rule: nothing fancy beyond vector add + recompute bounds.

Step 7 — when and how k-hop intra-cell refinement is triggered

This is the most important coupling between global and local.

You trigger local expansion when:

- v is a portal in some cell i, and
- the overlay currently has **insufficient** shortcut edges from v to other portals in i, or
- the search repeatedly hits "dead ends" around i, or
- v appears on/near the best solutions found so far.

Then call:

LocalExpand(cell i, seed portal p=v, hopLimit k, stepBudget B)

Local state and local labels

A local label is:

```pgsql
λ = {
  x: current base-graph vertex inside cell i,
  g[4]: accumulated vector cost from p to x (inside cell i only),
  hops: number of edges traversed,
```

```
    parent pointer (for reconstructing the intra-cell path)
  }
```

Copier le code

Local expansion rule (λ → λ2)

For each intra-cell edge (x→y) with y still in cell i:

$$g(\lambda_2) = g(\lambda) + c(x, y)$$

and `hops += 1`.

Stop expanding λ if `hops > k`.

Shortcut creation: portal-to-portal edges

Whenever local expansion reaches a portal q (q ≠ p):

- You have discovered a candidate **intra-cell shortcut** from p to q with vector cost `g(λ_at_q)`.
- You insert/update:

  `ShortcutStore[i][p][q] = best known (according to your shortcut ranking rule)`

Then you **materialize this shortcut into the overlay**:

- Add an overlay edge $p \to q$ with cost = `ShortcutStore[i][p][q].costVector`
- Store a pointer to the underlying base path for reconstruction.

This is exactly how "k-hop within a cell" produces new overlay edges.

Step 8 — how overlay costs are updated when refinement occurs

Two cases:

Case 1: new shortcut edge

You simply add a new overlay edge with its cost vector.

Case 2: improved shortcut edge (same (p,q) but better)

You replace or augment:

- If you keep multiple edges per pair (like COLA allows multiple representative edges), add it as another edge.
- If you keep one edge per pair, overwrite only if the new one is "better" under your shortcut criterion.

Overlay edges are allowed to represent paths in the base graph; this is standard overlay semantics. 2016 COLA Effective Indexing fo...

7) What is "Feasibility-first mode" exactly?

Because pruning becomes powerful once you have *any* feasible full path (it gives you UB^*), do an initial run that is biased toward feasibility:

Feasibility-first scoring for queue order:
Instead of ordering by LBscore(Chebyshev), order primarily by "distance to feasibility":

- Penalize violations of the box:
  - If g is below L, priority to increase those dimensions.
  - If g is above U, prune immediately (hard).
- Secondary key: cheapness / rough score.

This resonates with the general advice in doubly constrained methods: getting a strong **upper bound early** accelerates everything. 2024 A two-stage method for dou...

8) Does this ensure convergence to feasible solutions? What can break?

Convergence (feasible) — yes, under these "limit conditions"

If, across rounds, you ensure that:

1. **Portal set grows toward all boundary vertices** (eventually complete).
2. **k-hop limit grows to allow full intra-cell shortest/efficient connections** (eventually full).
3. **Compression/binning resolution tightens to exact** (eventually no approximation loss).
4. You don't prune anything that could be optimal except via admissible bounds.

Then in the limit, your overlay becomes "exact enough" and your label search becomes exact enough to find feasible solutions whenever they exist.

Practical gaps / inefficiencies to watch

1. **Lower-bound reachability intervals [LB_i,UB_i] are too weak**
   If your UB_i are huge, you won't prune "can't reach L" well. This causes label blowup (especially with lower constraints). This is a known pain point in min–max / doubly constrained search. 2024 A two-stage method for dou...
   Fix: maintain per-cell cheap summaries (min/max per-edge, plus "min-per-hop" conservative bounds) and refine bounds only near promising cells.

2. **Shortcut ranking rule can bias you away from feasibility**

   If you keep only "short" intra-cell shortcuts, you may fail to reach the *lower* bounds L.

   Fix: store at least 2–3 shortcut "types" per (p,q): one "cheap/short", one "resource-heavy" (useful to hit minima), one "balanced".

3. **Meeting point explosion in bidirectional join**

   If `BinsF[v]` and `BinsB[v]` each hold M labels, worst-case join cost is $O(M^2)$ per v.

   Fix: on join, pre-filter by quick feasibility interval tests, or keep bins in a structure keyed by normalized g to only try plausible complements. This mirrors bidirectional matching concerns in constrained A* frameworks. 2025 Resource Constrained Pathf...

4. **Cycle/elementarity (if you enforce it)**

   Your original plan didn't focus on elementarity, but if you later add "simple path only", explicit cycle checks can be costly; Min–Max RCSP work shows why this matters and motivates incremental frameworks that avoid heavy acyclicity checks.

   2025 Answering_Min-Max_Resource...

   Fix (if needed): cell-level taboo sets or ancestor-checking-like tricks, but only if you truly require elementarity.

9) Mermaid diagram of the runtime logic (main loop + local refinement)

mermaid

```
flowchart TD
  A[Start: OpenF={ℓ_s}, OpenB={ℓ_t}, UB*=+inf] --> B{while OpenF or OpenB not emp
  B --> C[Pick direction d with smallest key]
  C --> D[Pop ℓ from Open_d]
  D --> E{Box-feasible via reachability bounds?}
  E -- no --> B
  E -- yes --> F{LBscore(ℓ) < UB* ?}
  F -- no --> B
  F -- yes --> G{KeepInBins_d at node v=ℓ.v ?}
  G -- no --> H[Store ℓ in PrunedPool_d]
  H --> B
  G -- yes --> I[Bidirectional join at v with labels in Bins_opposite[v]]
  I --> J{Any feasible join improves archive?}
  J -- yes --> K[Update archive; update UB*]
  J -- no --> L[No update]
  K --> M[Expand ℓ over existing overlay edges -> push ℓ2 into Open_d]
  L --> M
```

```
M --> N{NeedMoreIntraCellEdges at portal v?}
N -- no --> B
N -- yes --> O[LocalExpand in cell(i), seed portal p=v, hopLimit k, stepBudget
O --> P[Update ShortcutStore; add/replace overlay shortcut edges]
P --> B
```

Diagramme Copier le code

If you want, I can also provide a **very explicit pseudocode block** for only these three routines (because they are the ones that usually cause confusion in implementation):

1. `KeepInBins()` (including bin index computation)
2. `Join(ℓ, ℓ_opp)` (including feasibility pruning before Score)
3. `LocalExpand()` (including how to update overlay edges safely)

Sources

JUst clarify in few sentences (not a lot) the l versus l2 or lambda versus lambda2 notation is it the portal versus intra celle difference ?

Good question — and no, it is **not** "portal vs intra-cell".
It is simply **parent state vs successor state**, at two different levels.

Very short clarification:

1 ℓ vs ℓ_2 (global / overlay level)

- ℓ = current *global label* at a **portal (overlay node)**
  → represents a partial path in the overlay graph.
- ℓ_2 = successor of ℓ after taking one **overlay edge**
  → created by:

$$g(\ell_2) = g(\ell) + c_{\text{overlay edge}}$$

So:

$\ell \rightarrow \ell_2$ is just one expansion step in the global (overlay) search.

Both ℓ and ℓ_2 live at **portals**.

2 λ vs λ_2 (local / intra-cell level)

- **λ** = current *local label* inside a **cell**
  → represents a partial path from a seed portal to some internal vertex.
- **λ_2** = successor of λ after taking one **intra-cell edge**
  → created by:

$$g(\lambda_2) = g(\lambda) + c_{\text{intra-cell edge}}$$

So:

λ → λ_2 is one expansion step inside a cell (k-hop exploration).

Both λ and λ_2 live on **base graph vertices inside a cell**.

Relationship between the two levels

When a local λ reaches another portal q:

- It produces a **shortcut edge (p → q)** in the overlay.
- That shortcut can then be used to create a new **global label ℓ_2**.

So the flow is:

sql

Copier le code

```
ℓ  (global at portal p)
   ↓ triggers local expansion
λ  (local inside cell)
   ↓ reaches portal q
create overlay edge (p→q)
   ↓
ℓ₂ = Extend(ℓ, p→q)   (global again)
```

In one sentence

- **ℓ / ℓ_2 = overlay (portal-level) search states**
- **λ / λ_2 = intra-cell (base-level) search states**
- The λ-layer *creates* overlay edges that ℓ-layer then uses.